

```
# ELIJAH SHE YU SHENG
# IC : 040413-13-1145

# Construct the data series with name data and use hierarchical
indexing where the output
# of the data set should be the same as in Figure 1.

# question a
import pandas as pd
data = pd.Series (
    [500,600,700,550,620,750],
    index = [['North','North','South','North','North','South'],
             ['West','East','Central','West','East','Central']]
)
data
```

North	West	500
	East	600
South	Central	700
North	West	550
	East	620
South	Central	750

dtype: int64

Rename the first index into Region and the second index to Sub-region. Rename the third column as Sales.

```
# question b
data.index.names = ['Region','Sub-region']
data.names = 'Sales'
print ("\nRenamed Indexes and Column: ")
data
```

Renamed Indexes and Column:

Region	Sub-region	
North	West	500
	East	600
South	Central	700
North	West	550
	East	620
South	Central	750

dtype: int64

Retrieve all sales values for the South region.

```
# question c
south_sale = data.loc['South']
```

```
print ("\nSales in South Region: ")
print (south_sale)
```

```
Sales in South Region:
Sub-region
Central      700
Central      750
dtype: int64
```

Calculate the total sales for each region

```
# question d
total_sale_region = data.groupby(level=0).sum() # region = level =0 //
'Region'
print ("\n Total sale by Region")
print (total_sale_region)
```

```
Total sale by Region
Region
North      2270
South      1450
dtype: int64
```

```
# question e
total_sale_subregion = data.groupby(level=1).sum() # region = level
=0 // 'Region'
print ("\n Total sale by Sub-region")
print (total_sale_subregion)
```

```
Total sale by Sub-region
Sub-region
Central      1450
East          1220
West          1050
dtype: int64
```

Convert the data series into a Pandas data frame with columns: "Region", "Sub-Region" and "Sales".

```
# question f (chap 6- reset index)
df = data.reset_index ()
df.columns = ['Region', 'Sub-region', 'Sales']
print ("\n Converted DataFrame:")
print (df)
```

```
Converted DataFrame:
```

	Region	Sub-region	Sales
0	North	West	500
1	North	East	600
2	South	Central	700
3	North	West	550
4	North	East	620
5	South	Central	750

Sort the data series in descending order of sales values and display the output.

```
# question g (Chap 4.2)
df_sort = df.sort_values (by= 'Sales',ascending = False)
print ("\nSorted data:")
print (df_sort)
```

Sorted data:

	Region	Sub-region	Sales
5	South	Central	750
2	South	Central	700
4	North	East	620
1	North	East	600
3	North	West	550
0	North	West	500

QUESTION 2

Answer the following questions to compute and rank U.S. state and territories by population density for the year 2010.

```
# question a (read file)
pop = pd.read_csv("state-population.csv")
areas = pd.read_csv("state-areas.csv")
abbrevs = pd.read_csv("state-abbrevs.csv")
```

*Merge the pop DataFrame with the abbrevs DataFrame to obtain the **full state name in the population dataset**. Use the **state/region** columns of **pop**, and the abbreviation columns of **abbrevs** as a key for merging. Use appropriate merge arguments to ensure no data is thrown away due to mismatched labels. Save the result as a new data frame named "merged".*

```
# question b : data wrangling (chapter 6- bottom)
merged = pop.merge (abbrevs,left_on ='state/region',right_on =
'abbreviation',
                    how = 'left')
merged
```

	state/region	ages	year	population	state	abbreviation
0	AL	under18	2012	1117489.0	Alabama	AL
1	AL	total	2012	4817528.0	Alabama	AL

2	AL	under18	2010	1130966.0	Alabama	AL
3	AL	total	2010	4785570.0	Alabama	AL
4	AL	under18	2011	1125763.0	Alabama	AL
...	...	...	...	...	...	...
2539	USA	total	2010	309326295.0	NaN	NaN
2540	USA	under18	2011	73902222.0	NaN	NaN
2541	USA	total	2011	311582564.0	NaN	NaN
2542	USA	under18	2012	73708179.0	NaN	NaN
2543	USA	total	2012	313873685.0	NaN	NaN

[2544 rows x 6 columns]

Identify and remove any duplicated columns in the merged data frame.

```
# question c - remove duplicated column
merged = merged.loc[:,~merged.columns.duplicated()]
merged
```

	state/region	ages	year	population	state	abbreviation
0	AL	under18	2012	1117489.0	Alabama	AL
1	AL	total	2012	4817528.0	Alabama	AL
2	AL	under18	2010	1130966.0	Alabama	AL
3	AL	total	2010	4785570.0	Alabama	AL
4	AL	under18	2011	1125763.0	Alabama	AL
...	...	...	...	...	...	...
2539	USA	total	2010	309326295.0	NaN	NaN
2540	USA	under18	2011	73902222.0	NaN	NaN
2541	USA	total	2011	311582564.0	NaN	NaN
2542	USA	under18	2012	73708179.0	NaN	NaN
2543	USA	total	2012	313873685.0	NaN	NaN

[2544 rows x 6 columns]

Check for missing values. List the columns that contain the missing values thus remove all rows that contain missing values.

```
# question d (check missing value + remove missing value row)
missing_values = merged.isnull().sum()# total up the column
print (missing_values)
print (missing_values[missing_values>0])
```

```
state/region      0
ages              0
year              0
population        20
state             96
abbreviation      96
dtype: int64
population        20
```

```
state          96
abbreviation    96
dtype: int64
```

Merge the cleaned merged data frame with the **area data frame**. Join the data frame using the **state columns** as a common key. Save the final combined dataset as **final**. Display the **top 5 rows** of the final data frame.

```
#question e (head top 5 )
final = merged.merge (areas,on ='state',how ='left') # on = 并列, how =
take left
print (final.head())
```

	state/region	ages	year	population	state	abbreviation	area
0	AL	under18	2012	1117489.0	Alabama	AL	52423.0
1	AL	total	2012	4817528.0	Alabama	AL	52423.0
2	AL	under18	2010	1130966.0	Alabama	AL	52423.0
3	AL	total	2010	4785570.0	Alabama	AL	52423.0
4	AL	under18	2011	1125763.0	Alabama	AL	52423.0

Remove any rows with missing values from the final data frame.

```
#question f (remove missing value)
final.dropna(inplace =True)
final
#final[final.notnull()]
```

	state/region	ages	year	population	state	abbreviation	\
0	AL	under18	2012	1117489.0	Alabama	AL	
1	AL	total	2012	4817528.0	Alabama	AL	
2	AL	under18	2010	1130966.0	Alabama	AL	
3	AL	total	2010	4785570.0	Alabama	AL	
4	AL	under18	2011	1125763.0	Alabama	AL	
...	...	...	...	...	...	...	...
2443	WY	under18	1993	137458.0	Wyoming	WY	
2444	WY	total	1991	459260.0	Wyoming	WY	
2445	WY	under18	1991	136720.0	Wyoming	WY	
2446	WY	under18	1990	136078.0	Wyoming	WY	
2447	WY	total	1990	453690.0	Wyoming	WY	

	area (sq. mi)
0	52423.0
1	52423.0

```

2          52423.0
3          52423.0
4          52423.0
...
2443       97818.0
2444       97818.0
2445       97818.0
2446       97818.0
2447       97818.0

```

```
[2448 rows x 7 columns]
```

Filter the data to include **only rows corresponding to 2010** and the **total age group** using **query()** function as follows.

```
Data2010=final.query("year == 2010 & ages =='total'")
Data2010
```

	state/region abbreviation \	ages	year	population	state
3	AL	total	2010	4785570.0	Alabama
AL					
91	AK	total	2010	713868.0	Alaska
AK					
101	AZ	total	2010	6408790.0	Arizona
AZ					
189	AR	total	2010	2922280.0	Arkansas
AR					
197	CA	total	2010	37333601.0	California
CA					
283	CO	total	2010	5048196.0	Colorado
CO					
293	CT	total	2010	3579210.0	Connecticut
CT					
379	DE	total	2010	899711.0	Delaware
DE					
389	DC	total	2010	605125.0	District of Columbia
DC					
475	FL	total	2010	18846054.0	Florida
FL					
485	GA	total	2010	9713248.0	Georgia
GA					
570	HI	total	2010	1363731.0	Hawaii
HI					
581	ID	total	2010	1570718.0	Idaho
ID					
666	IL	total	2010	12839695.0	Illinois
IL					
677	IN	total	2010	6489965.0	Indiana

IN					
762	IA	total	2010	3050314.0	Iowa
IA					
773	KS	total	2010	2858910.0	Kansas
KS					
858	KY	total	2010	4347698.0	Kentucky
KY					
869	LA	total	2010	4545392.0	Louisiana
LA					
954	ME	total	2010	1327366.0	Maine
ME					
965	MD	total	2010	5787193.0	Maryland
MD					
1050	MA	total	2010	6563263.0	Massachusetts
MA					
1061	MI	total	2010	9876149.0	Michigan
MI					
1146	MN	total	2010	5310337.0	Minnesota
MN					
1157	MS	total	2010	2970047.0	Mississippi
MS					
1242	MO	total	2010	5996063.0	Missouri
MO					
1253	MT	total	2010	990527.0	Montana
MT					
1338	NE	total	2010	1829838.0	Nebraska
NE					
1349	NV	total	2010	2703230.0	Nevada
NV					
1434	NH	total	2010	1316614.0	New Hampshire
NH					
1445	NJ	total	2010	8802707.0	New Jersey
NJ					
1530	NM	total	2010	2064982.0	New Mexico
NM					
1541	NY	total	2010	19398228.0	New York
NY					
1626	NC	total	2010	9559533.0	North Carolina
NC					
1637	ND	total	2010	674344.0	North Dakota
ND					
1722	OH	total	2010	11545435.0	Ohio
OH					
1733	OK	total	2010	3759263.0	Oklahoma
OK					
1818	OR	total	2010	3837208.0	Oregon
OR					
1829	PA	total	2010	12710472.0	Pennsylvania
PA					

1914	RI	total	2010	1052669.0	Rhode Island
RI					
1925	SC	total	2010	4636361.0	South Carolina
SC					
2010	SD	total	2010	816211.0	South Dakota
SD					
2021	TN	total	2010	6356683.0	Tennessee
TN					
2106	TX	total	2010	25245178.0	Texas
TX					
2117	UT	total	2010	2774424.0	Utah
UT					
2202	VT	total	2010	625793.0	Vermont
VT					
2213	VA	total	2010	8024417.0	Virginia
VA					
2298	WA	total	2010	6742256.0	Washington
WA					
2309	WV	total	2010	1854146.0	West Virginia
WV					
2394	WI	total	2010	5689060.0	Wisconsin
WI					
2405	WY	total	2010	564222.0	Wyoming
WY					

	area (sq. mi)
3	52423.0
91	656425.0
101	114006.0
189	53182.0
197	163707.0
283	104100.0
293	5544.0
379	1954.0
389	68.0
475	65758.0
485	59441.0
570	10932.0
581	83574.0
666	57918.0
677	36420.0
762	56276.0
773	82282.0
858	40411.0
869	51843.0
954	35387.0
965	12407.0
1050	10555.0
1061	96810.0

1146	86943.0
1157	48434.0
1242	69709.0
1253	147046.0
1338	77358.0
1349	110567.0
1434	9351.0
1445	8722.0
1530	121593.0
1541	54475.0
1626	53821.0
1637	70704.0
1722	44828.0
1733	69903.0
1818	98386.0
1829	46058.0
1914	1545.0
1925	32007.0
2010	77121.0
2021	42146.0
2106	268601.0
2117	84904.0
2202	9615.0
2213	42769.0
2298	71303.0
2309	24231.0
2394	65503.0
2405	97818.0

Set the state as a new index for Data2010 data frame

```
#question 2a (set index _data wrangling)- state = new index
#Data2010.set_index('state',inplace= True)
Data2010
```

	state/region	ages	year	population
abbreviation \				
state				
Alabama	AL	total	2010	4785570.0
AL				
Alaska	AK	total	2010	713868.0
AK				
Arizona	AZ	total	2010	6408790.0
AZ				
Arkansas	AR	total	2010	2922280.0
AR				
California	CA	total	2010	37333601.0
CA				

Colorado	CO	total	2010	5048196.0
CO				
Connecticut	CT	total	2010	3579210.0
CT				
Delaware	DE	total	2010	899711.0
DE				
District of Columbia	DC	total	2010	605125.0
DC				
Florida	FL	total	2010	18846054.0
FL				
Georgia	GA	total	2010	9713248.0
GA				
Hawaii	HI	total	2010	1363731.0
HI				
Idaho	ID	total	2010	1570718.0
ID				
Illinois	IL	total	2010	12839695.0
IL				
Indiana	IN	total	2010	6489965.0
IN				
Iowa	IA	total	2010	3050314.0
IA				
Kansas	KS	total	2010	2858910.0
KS				
Kentucky	KY	total	2010	4347698.0
KY				
Louisiana	LA	total	2010	4545392.0
LA				
Maine	ME	total	2010	1327366.0
ME				
Maryland	MD	total	2010	5787193.0
MD				
Massachusetts	MA	total	2010	6563263.0
MA				
Michigan	MI	total	2010	9876149.0
MI				
Minnesota	MN	total	2010	5310337.0
MN				
Mississippi	MS	total	2010	2970047.0
MS				
Missouri	MO	total	2010	5996063.0
MO				
Montana	MT	total	2010	990527.0
MT				
Nebraska	NE	total	2010	1829838.0
NE				
Nevada	NV	total	2010	2703230.0
NV				
New Hampshire	NH	total	2010	1316614.0

NH					
New Jersey	NJ	total	2010	8802707.0	
NJ					
New Mexico	NM	total	2010	2064982.0	
NM					
New York	NY	total	2010	19398228.0	
NY					
North Carolina	NC	total	2010	9559533.0	
NC					
North Dakota	ND	total	2010	674344.0	
ND					
Ohio	OH	total	2010	11545435.0	
OH					
Oklahoma	OK	total	2010	3759263.0	
OK					
Oregon	OR	total	2010	3837208.0	
OR					
Pennsylvania	PA	total	2010	12710472.0	
PA					
Rhode Island	RI	total	2010	1052669.0	
RI					
South Carolina	SC	total	2010	4636361.0	
SC					
South Dakota	SD	total	2010	816211.0	
SD					
Tennessee	TN	total	2010	6356683.0	
TN					
Texas	TX	total	2010	25245178.0	
TX					
Utah	UT	total	2010	2774424.0	
UT					
Vermont	VT	total	2010	625793.0	
VT					
Virginia	VA	total	2010	8024417.0	
VA					
Washington	WA	total	2010	6742256.0	
WA					
West Virginia	WV	total	2010	1854146.0	
WV					
Wisconsin	WI	total	2010	5689060.0	
WI					
Wyoming	WY	total	2010	564222.0	
WY					

	area (sq. mi)
state	
Alabama	52423.0
Alaska	656425.0
Arizona	114006.0

Arkansas	53182.0
California	163707.0
Colorado	104100.0
Connecticut	5544.0
Delaware	1954.0
District of Columbia	68.0
Florida	65758.0
Georgia	59441.0
Hawaii	10932.0
Idaho	83574.0
Illinois	57918.0
Indiana	36420.0
Iowa	56276.0
Kansas	82282.0
Kentucky	40411.0
Louisiana	51843.0
Maine	35387.0
Maryland	12407.0
Massachusetts	10555.0
Michigan	96810.0
Minnesota	86943.0
Mississippi	48434.0
Missouri	69709.0
Montana	147046.0
Nebraska	77358.0
Nevada	110567.0
New Hampshire	9351.0
New Jersey	8722.0
New Mexico	121593.0
New York	54475.0
North Carolina	53821.0
North Dakota	70704.0
Ohio	44828.0
Oklahoma	69903.0
Oregon	98386.0
Pennsylvania	46058.0
Rhode Island	1545.0
South Carolina	32007.0
South Dakota	77121.0
Tennessee	42146.0
Texas	268601.0
Utah	84904.0
Vermont	9615.0
Virginia	42769.0
Washington	71303.0
West Virginia	24231.0
Wisconsin	65503.0
Wyoming	97818.0

Calculate the population density by using this formula.

#question 2b

```
Data2010["Density"] = Data2010["population"] / Data2010["area (sq. mi)"]
```

C:\Users\User\AppData\Local\Temp\ipykernel_18544\3013420272.py:2:

SettingWithCopyWarning:

A value is trying to be set on a copy of a slice from a DataFrame.

Try using `.loc[row_indexer,col_indexer] = value` instead

See the caveats in the documentation:

https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy

```
Data2010["Density"] = Data2010["population"] / Data2010["area (sq. mi)"]
```

Sort the population density in descending order.

question 2c

```
Data2010_sorted = Data2010.sort_values(by="Density", ascending = False)
```

```
Data2010_sorted
```

abbreviation \ state	state/region	ages	year	population
District of Columbia DC	DC	total	2010	605125.0
New Jersey NJ	NJ	total	2010	8802707.0
Rhode Island RI	RI	total	2010	1052669.0
Connecticut CT	CT	total	2010	3579210.0
Massachusetts MA	MA	total	2010	6563263.0
Maryland MD	MD	total	2010	5787193.0
Delaware DE	DE	total	2010	899711.0
New York NY	NY	total	2010	19398228.0
Florida FL	FL	total	2010	18846054.0
Pennsylvania PA	PA	total	2010	12710472.0
Ohio OH	OH	total	2010	11545435.0
California CA	CA	total	2010	37333601.0

Illinois	IL	total	2010	12839695.0
IL				
Virginia	VA	total	2010	8024417.0
VA				
Indiana	IN	total	2010	6489965.0
IN				
North Carolina	NC	total	2010	9559533.0
NC				
Georgia	GA	total	2010	9713248.0
GA				
Tennessee	TN	total	2010	6356683.0
TN				
South Carolina	SC	total	2010	4636361.0
SC				
New Hampshire	NH	total	2010	1316614.0
NH				
Hawaii	HI	total	2010	1363731.0
HI				
Kentucky	KY	total	2010	4347698.0
KY				
Michigan	MI	total	2010	9876149.0
MI				
Washington	WA	total	2010	6742256.0
WA				
Texas	TX	total	2010	25245178.0
TX				
Alabama	AL	total	2010	4785570.0
AL				
Louisiana	LA	total	2010	4545392.0
LA				
Wisconsin	WI	total	2010	5689060.0
WI				
Missouri	MO	total	2010	5996063.0
MO				
West Virginia	WV	total	2010	1854146.0
WV				
Vermont	VT	total	2010	625793.0
VT				
Mississippi	MS	total	2010	2970047.0
MS				
Minnesota	MN	total	2010	5310337.0
MN				
Arizona	AZ	total	2010	6408790.0
AZ				
Arkansas	AR	total	2010	2922280.0
AR				
Iowa	IA	total	2010	3050314.0
IA				
Oklahoma	OK	total	2010	3759263.0

OK				
Colorado	CO	total	2010	5048196.0
CO				
Oregon	OR	total	2010	3837208.0
OR				
Maine	ME	total	2010	1327366.0
ME				
Kansas	KS	total	2010	2858910.0
KS				
Utah	UT	total	2010	2774424.0
UT				
Nevada	NV	total	2010	2703230.0
NV				
Nebraska	NE	total	2010	1829838.0
NE				
Idaho	ID	total	2010	1570718.0
ID				
New Mexico	NM	total	2010	2064982.0
NM				
South Dakota	SD	total	2010	816211.0
SD				
North Dakota	ND	total	2010	674344.0
ND				
Montana	MT	total	2010	990527.0
MT				
Wyoming	WY	total	2010	564222.0
WY				
Alaska	AK	total	2010	713868.0
AK				

	area (sq. mi)	Density
state		
District of Columbia	68.0	8898.897059
New Jersey	8722.0	1009.253268
Rhode Island	1545.0	681.339159
Connecticut	5544.0	645.600649
Massachusetts	10555.0	621.815538
Maryland	12407.0	466.445797
Delaware	1954.0	460.445752
New York	54475.0	356.094135
Florida	65758.0	286.597129
Pennsylvania	46058.0	275.966651
Ohio	44828.0	257.549634
California	163707.0	228.051342
Illinois	57918.0	221.687472
Virginia	42769.0	187.622273
Indiana	36420.0	178.197831
North Carolina	53821.0	177.617157
Georgia	59441.0	163.409902

Tennessee	42146.0	150.825298
South Carolina	32007.0	144.854594
New Hampshire	9351.0	140.799273
Hawaii	10932.0	124.746707
Kentucky	40411.0	107.586994
Michigan	96810.0	102.015794
Washington	71303.0	94.557817
Texas	268601.0	93.987655
Alabama	52423.0	91.287603
Louisiana	51843.0	87.676099
Wisconsin	65503.0	86.851900
Missouri	69709.0	86.015622
West Virginia	24231.0	76.519582
Vermont	9615.0	65.085075
Mississippi	48434.0	61.321530
Minnesota	86943.0	61.078373
Arizona	114006.0	56.214497
Arkansas	53182.0	54.948667
Iowa	56276.0	54.202751
Oklahoma	69903.0	53.778278
Colorado	104100.0	48.493718
Oregon	98386.0	39.001565
Maine	35387.0	37.509990
Kansas	82282.0	34.745266
Utah	84904.0	32.677188
Nevada	110567.0	24.448796
Nebraska	77358.0	23.654153
Idaho	83574.0	18.794338
New Mexico	121593.0	16.982737
South Dakota	77121.0	10.583512
North Dakota	70704.0	9.537565
Montana	147046.0	6.736171
Wyoming	97818.0	5.768079
Alaska	656425.0	1.087509

Display the top 5 least densely populated states.

```
# question 2d(top 5 least)
print (Data2010_sorted.tail(5))
```

state	state/region	ages	year	population	abbreviation	\
South Dakota	SD	total	2010	816211.0	SD	
North Dakota	ND	total	2010	674344.0	ND	
Montana	MT	total	2010	990527.0	MT	
Wyoming	WY	total	2010	564222.0	WY	
Alaska	AK	total	2010	713868.0	AK	

area (sq. mi)	Density
---------------	---------

state		
South Dakota	77121.0	10.583512
North Dakota	70704.0	9.537565
Montana	147046.0	6.736171
Wyoming	97818.0	5.768079
Alaska	656425.0	1.087509

QUESTION 3

```
#Load the dataset employee_performance_data.csv.
#Set the Employee_ID column as the index.
# question a
import pandas as pd
emp_performance = pd.read_csv ("employee_performance_data.csv",
                               index_col = "Employee_ID")
emp_details =pd.read_csv("employee_details.csv")
emp_performance
```

	Name	Department	Monthly_Salary	Years_at_Company
\				
Employee_ID				
101	Employee 1	Marketing	6021	6
102	Employee 2	HR	4073	3
103	Employee 3	Sales	7613	12
104	Employee 4	IT	4276	1
105	Employee 5	IT	7720	7
106	Employee 6	HR	5202	13
107	Employee 7	HR	4625	10
108	Employee 8	Finance	3034	10
109	Employee 9	Marketing	4408	1
110	Employee 10	Marketing	3817	11
111	Employee 11	IT	6179	6
112	Employee 12	Sales	6566	5
113	Employee 13	Marketing	7549	7
114	Employee 14	IT	6272	8
115	Employee 15	IT	6690	7

116	Employee 16	Sales	7029	15
117	Employee 17	Marketing	5145	15
118	Employee 18	Sales	5184	6
119	Employee 19	IT	3951	11
120	Employee 20	Sales	6462	1
121	Employee 21	HR	5265	3
122	Employee 22	HR	7776	11
123	Employee 23	Marketing	3914	2
124	Employee 24	Sales	5050	7
125	Employee 25	Sales	3282	15
126	Employee 26	HR	7056	12
127	Employee 27	HR	5287	11
128	Employee 28	Sales	5820	14
129	Employee 29	Sales	6966	12
130	Employee 30	HR	7120	9
Employee_ID	Performance_Score	Last_Promotion_Year	Work_Location	
101	4.7	2024	Rural	
102	3.1	2021	Urban	
103	3.7	2024	Rural	
104	4.3	2016	Urban	
105	4.0	2018	Rural	
106	4.6	2023	Rural	
107	5.0	2023	Rural	
108	3.8	2022	Rural	
109	2.9	2021	Urban	
110	3.4	2022	Urban	
111	3.1	2019	Urban	
112	5.0	2021	Rural	
113	2.6	2017	Rural	
114	4.7	2022	Rural	
115	3.6	2019	Rural	
116	3.6	2024	Urban	
117	4.3	2019	Urban	

118	5.0	2019	Rural
119	3.5	2017	Rural
120	4.1	2016	Rural
121	2.9	2020	Rural
122	4.0	2023	Urban
123	3.4	2024	Rural
124	3.8	2024	Rural
125	4.2	2023	Urban
126	4.6	2024	Rural
127	2.7	2023	Rural
128	4.9	2021	Rural
129	3.5	2022	Rural
130	3.2	2017	Rural

*#Extract and display all records for employees who:
(3 marks)*

-->Work in the Sales department.

-->Have a Performance_Score greater than 4

question b

```
high_perf_sales = emp_performance [(emp_performance["Department"] ==
"Sales")&
```

```
(emp_performance["Performance_Score"]> 4) ]
```

```
high_perf_sales
```

	Name	Department	Monthly_Salary	Years_at_Company
\				
Employee_ID				
112	Employee 12	Sales	6566	5
118	Employee 18	Sales	5184	6
120	Employee 20	Sales	6462	1
125	Employee 25	Sales	3282	15
128	Employee 28	Sales	5820	14

	Performance_Score	Last_Promotion_Year	Work_Location
Employee_ID			
112	5.0	2021	Rural
118	5.0	2019	Rural
120	4.1	2016	Rural
125	4.2	2023	Urban
128	4.9	2021	Rural

Merge employee_performance_data.csv

and employee_details.csv using the Employee_ID column.

question c

```
merge_emp =
emp_performance.reset_index().merge(emp_details,on="Employee_ID")
print (merge_emp)
```

	Employee_ID	Name	Department	Monthly_Salary
Years_at_Company \				
0	101	Employee 1	Marketing	6021
6				
1	102	Employee 2	HR	4073
3				
2	103	Employee 3	Sales	7613
12				
3	104	Employee 4	IT	4276
1				
4	105	Employee 5	IT	7720
7				
5	106	Employee 6	HR	5202
13				
6	107	Employee 7	HR	4625
10				

	Performance_Score	Last_Promotion_Year	Work_Location
0	4.7	2024	Rural
1	3.1	2021	Urban
2	3.7	2024	Rural
3	4.3	2016	Urban
4	4.0	2018	Rural
5	4.6	2023	Rural
6	5.0	2023	Rural

	Email	Phone_Number
0	alice.johnson@example.com	1234567890
1	bob.smith@example.com	2345678901
2	cathy.lee@example.com	3456789012
3	david.brown@example.com	4567890123
4	emma.wilson@example.com	5678901234
5	frank.knight@example.com	6789012345
6	grace.white@example.com	7890123456

```
# Filter employees with a Monthly_Salary greater than 4500
# and store into high_salary_employees variable.
# question d
high_salary_employees =merge_emp [(merge_emp["Monthly_Salary"]>4500)]
high_salary_employees
```

	Employee_ID	Name	Department	Monthly_Salary
Years_at_Company \				
0	101	Employee 1	Marketing	6021
6				
2	103	Employee 3	Sales	7613

```

12
4      105  Employee 5      IT      7720
7
5      106  Employee 6      HR      5202
13
6      107  Employee 7      HR      4625
10

```

```

      Performance_Score  Last_Promotion_Year  Work_Location \
0          4.7          2024          Rural
2          3.7          2024          Rural
4          4.0          2018          Rural
5          4.6          2023          Rural
6          5.0          2023          Rural

```

```

      Email  Phone_Number
0  alice.johnson@example.com  1234567890
2   cathy.lee@example.com    3456789012
4   emma.wilson@example.com  5678901234
5  frank.knight@example.com  6789012345
6  grace.white@example.com   7890123456

```

Combine their email addresses into a list email_list and display.

question e

```
email_list = high_salary_employees["Email"].tolist()
```

```
email_list
```

```

['alice.johnson@example.com',
 'cathy.lee@example.com',
 'emma.wilson@example.com',
 'frank.knight@example.com',
 'grace.white@example.com']

```

Sort the merged dataset by Department

and then by Performance_Score in descending order.

question f

```

sort_emp = merge_emp.sort_values(by
=[ "Department", "Performance_Score"],
                                ascending =[True, False])

```

```
print (sort_emp)
```

```

      Employee_ID      Name  Department  Monthly_Salary
Years_at_Company \
6      107  Employee 7      HR      4625
10
5      106  Employee 6      HR      5202
13
1      102  Employee 2      HR      4073
3
3      104  Employee 4      IT      4276

```

```

1
4      105 Employee 5      IT      7720
7
0      101 Employee 1 Marketing      6021
6
2      103 Employee 3      Sales      7613
12

```

```

      Performance_Score Last_Promotion_Year Work_Location \
6      5.0      2023      Rural
5      4.6      2023      Rural
1      3.1      2021      Urban
3      4.3      2016      Urban
4      4.0      2018      Rural
0      4.7      2024      Rural
2      3.7      2024      Rural

```

```

      Email Phone_Number
6  grace.white@example.com 7890123456
5  frank.knight@example.com 6789012345
1  bob.smith@example.com 2345678901
3  david.brown@example.com 4567890123
4  emma.wilson@example.com 5678901234
0  alice.johnson@example.com 1234567890
2  cathy.lee@example.com 3456789012

```

```

# Reorder the dataset columns as follows:
# Name, Department, Monthly_Salary, Performance_Score, Email,
# Phone_Number.
# question g
sort_emp = sort_emp [['Name', 'Department', 'Monthly_Salary',
                        'Performance_Score', 'Email',
                        'Phone_Number', 'Last_Promotion_Year']]
print(sort_emp)

```

```

      Name Department Monthly_Salary Performance_Score \
6 Employee 7      HR      4625      5.0
5 Employee 6      HR      5202      4.6
1 Employee 2      HR      4073      3.1
3 Employee 4      IT      4276      4.3
4 Employee 5      IT      7720      4.0
0 Employee 1 Marketing      6021      4.7
2 Employee 3      Sales      7613      3.7

```

```

      Email Phone_Number Last_Promotion_Year
6  grace.white@example.com 7890123456      2023
5  frank.knight@example.com 6789012345      2023
1  bob.smith@example.com 2345678901      2021
3  david.brown@example.com 4567890123      2016
4  emma.wilson@example.com 5678901234      2018

```

```
0  alice.johnson@example.com  1234567890  2024
2  cathy.lee@example.com  3456789012  2024
```

```
# Calculate the difference between 2024 and the Last_Promotion_Year
# question h
```

```
sort_emp ["difference_Promotion_Year"] = 2025 -
sort_emp["Last_Promotion_Year"]
print (sort_emp)
```

	Name	Department	Monthly_Salary	Performance_Score	\
6	Employee 7	HR	4625	5.0	
5	Employee 6	HR	5202	4.6	
1	Employee 2	HR	4073	3.1	
3	Employee 4	IT	4276	4.3	
4	Employee 5	IT	7720	4.0	
0	Employee 1	Marketing	6021	4.7	
2	Employee 3	Sales	7613	3.7	

	Email	Phone_Number	Last_Promotion_Year	\
6	grace.white@example.com	7890123456	2023	
5	frank.knight@example.com	6789012345	2023	
1	bob.smith@example.com	2345678901	2021	
3	david.brown@example.com	4567890123	2016	
4	emma.wilson@example.com	5678901234	2018	
0	alice.johnson@example.com	1234567890	2024	
2	cathy.lee@example.com	3456789012	2024	

	difference_Promotion_Year
6	2
5	2
1	4
3	9
4	7
0	1
2	1

```
# Create a new column, Promotion_Status with the following status
values:
```

```
# "Recently Promoted": Years_Since_Last_Promotion <= 2.
```

```
# "Due for Promotion": Years_Since_Last_Promotion > 5 and
Performance_Score >= 4.
```

```
# "No Promotion Needed": All other cases.
```

```
# question i (chapter 7)
```

```
def promotion_status(row):
    if row["difference_Promotion_Year"] <= 2 : return "Recently
Promoted"
    elif row["difference_Promotion_Year"] > 5 and
row["difference_Promotion_Year"] >= 4: return "Due for Promotion"
    else : return "No Promotion Needed"
```

```
sort_emp["Promotion_Status"] = sort_emp.apply(promotion_status,
axis=1)
print (sort_emp)
```

	Name	Department	Monthly_Salary	Performance_Score	\
6	Employee 7	HR	4625	5.0	
5	Employee 6	HR	5202	4.6	
1	Employee 2	HR	4073	3.1	
3	Employee 4	IT	4276	4.3	
4	Employee 5	IT	7720	4.0	
0	Employee 1	Marketing	6021	4.7	
2	Employee 3	Sales	7613	3.7	

	Email	Phone_Number	Last_Promotion_Year	\
6	grace.white@example.com	7890123456	2023	
5	frank.knight@example.com	6789012345	2023	
1	bob.smith@example.com	2345678901	2021	
3	david.brown@example.com	4567890123	2016	
4	emma.wilson@example.com	5678901234	2018	
0	alice.johnson@example.com	1234567890	2024	
2	cathy.lee@example.com	3456789012	2024	

	difference_Promotion_Year	Promotion_Status
6	2	Recently Promoted
5	2	Recently Promoted
1	4	No Promotion Needed
3	9	Due for Promotion
4	7	Due for Promotion
0	1	Recently Promoted
2	1	Recently Promoted

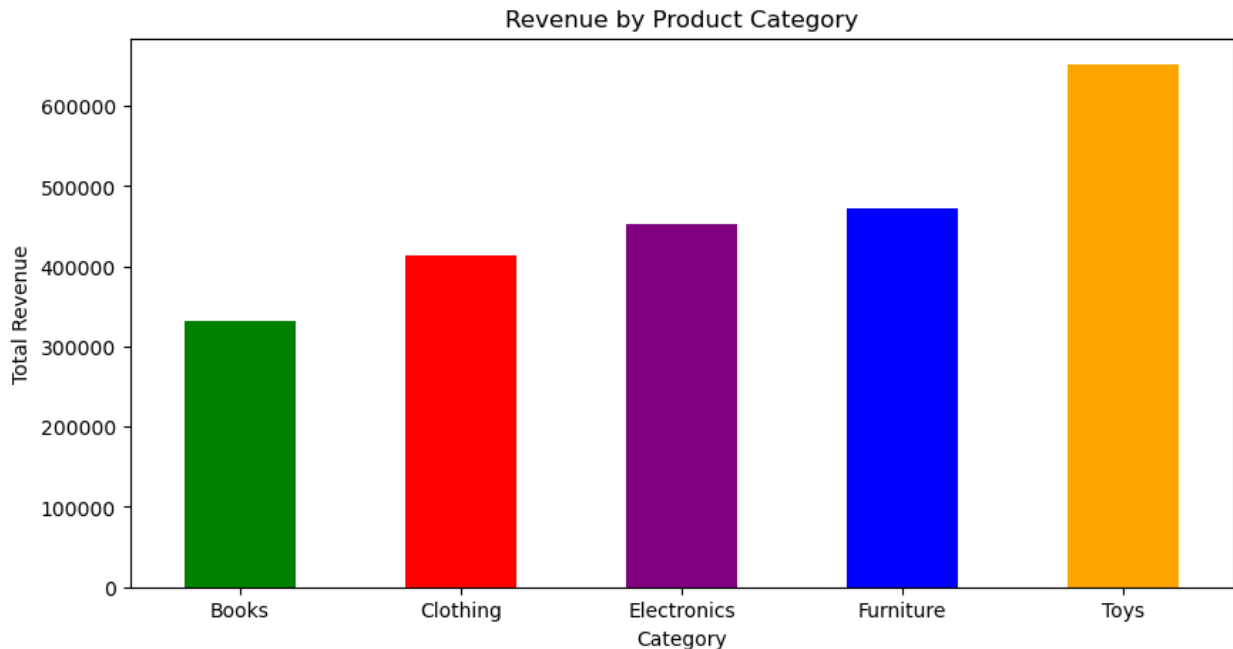
QUESTION 4 Use the dataset products_sales.csv to answer the following questions. Your visualizations must use only the Matplotlib library. Dataset Columns:

--> **Product_ID**: Unique identifier for products. --> **Category**: Product category (e.g., Electronics, Furniture). --> **Region**: Sales region (e.g., North, South). --> **Units_Sold**: Number of units sold. --> **Revenue**: Total revenue generated. --> **Profit**: Total profit earned. --> **Month**: Month of the sale (e.g., January, February).

```
# Create a bar chart to show the total revenue generated by each
product category.
# Apply the following formatting to produce chart as in Figure 1
example: (8 marks)
# title: "Revenue by Product Category".
# X-axis label: "Category".
# Y-axis label: "Total Revenue".
# Use a unique color for each bar.
# question a
import matplotlib.pyplot as plt
sales_data = pd.read_csv("products_sales.csv")
```



```
category_revenue = sales_data.groupby("Category")["Revenue"].sum()
plt.figure(figsize=(10,5))
category_revenue.plot(kind="bar",color=['green','red',"purple","blue",
"orange"])
plt.title("Revenue by Product Category")
plt.xlabel("Category")
plt.ylabel("Total Revenue")
plt.xticks(rotation =0)
plt.show()
```



```
# Which category has the highest total revenue?
# question b
top_category = category_revenue.idxmax()
print ("the highest total revenue: ",top_category)
```

the highest total revenue: Toys

```
# Is there any significant difference between the revenues across the
categories?
print ("Revenue by Category:\n",category_revenue)
```

```
Revenue by Category:
Category
Books      331317
Clothing   413142
Electronics 452286
Furniture  472348
Toys       651777
Name: Revenue, dtype: int64
```

```
# Create a line plot to show the trend of total revenue across months
for the top-performing region with the following formatting:
(8 marks)
```

```
# Title: "Monthly Revenue Trend for [Top Region]".
```

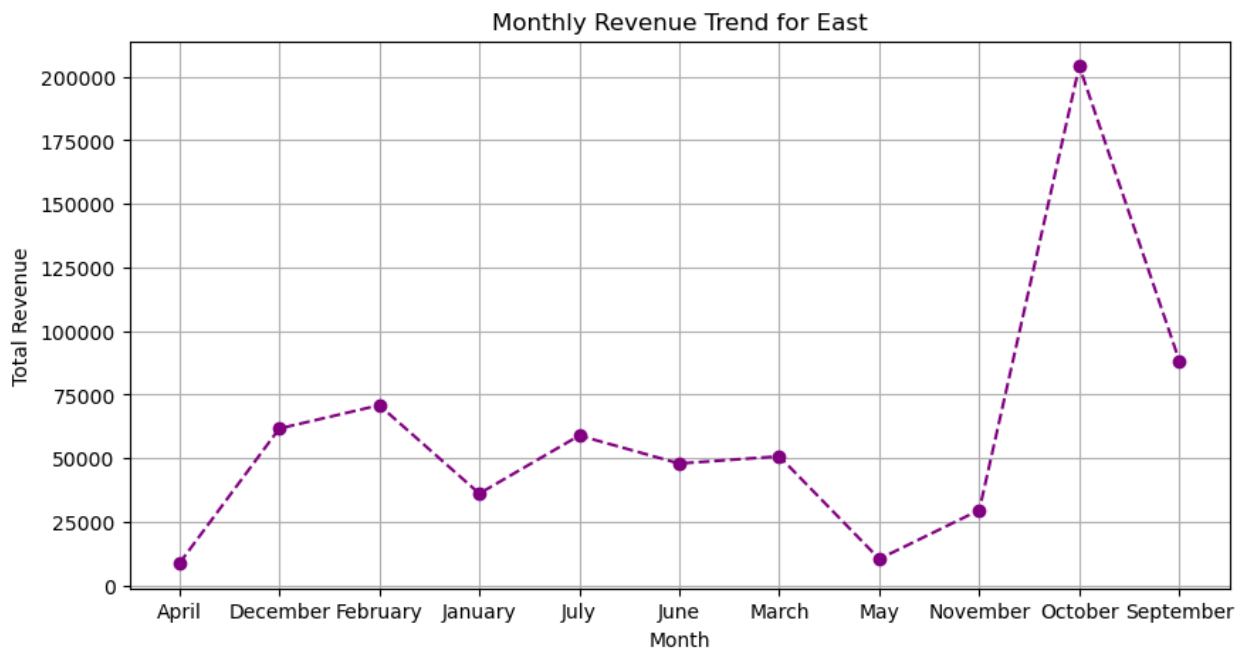
```
# X-axis label: "Month".
```

```
# Y-axis label: "Total Revenue".
```

```
# Add markers at each data point and use a dashed line style.
```

```
top_region = sales_data.groupby("Region")["Revenue"].sum().idxmax()
monthly_revenue = top_region_data.groupby("Month")["Revenue"].sum()
```

```
plt.figure(figsize=(10,5))
plt.plot(monthly_revenue.index, monthly_revenue.values,
marker="o",linestyle='dashed',color = 'purple')
plt.title(f"Monthly Revenue Trend for {top_region}")
plt.xlabel("Month")
plt.ylabel("Total Revenue")
plt.grid(True)
plt.show()
```



```
# Identify which quarter has the highest revenue and which quarter has
the lowest?
```

```
sales_data["Quarter"] = pd.to_datetime(sales_data["Month"],format
="%B").dt.quarter
```

```
quarterly_revenue = sales_data.groupby("Quarter")["Revenue"].sum()
```

```
highest_quarter = quarterly_revenue.idxmax()
```

```
lowest_quarter = quarterly_revenue.idxmin()
```

```
print(f"Highest revenue quarter: Q{highest_quarter}")  
print(f"Lowest revenue quarter: Q{lowest_quarter}")
```

```
Highest revenue quarter: Q4  
Lowest revenue quarter: Q1
```